Senarath Bank - Security Simulation Lab

Full Project Documentation

Deliberately vulnerable web applications for CrowdStrike NG SIEM detection training · All data fictional · 2026-10-28

Status: Training lab · **Type:** Deliberately vulnerable web applications **Purpose:** Generate realistic attack + audit telemetry for CrowdStrike NG SIEM detection training
Last updated: 2026-10-28

This is a simulation. *Senarath Bank* is fictional; all customers, accounts, balances and transactions are seeded fake data. Both apps are intentionally insecure. Never place real people, real credentials, real money or production data in here, and run it only on an isolated network.

---

1. Overview

This repository contains the two **DMZ-1** web applications from the CrowdStrike NG SIEM reference architecture, plus the shared database tier and the log layout the on-prem NG SIEM Log Collector consumes.

Component	Architecture role	Folder	Host port	Container
General website + CMS	DMZ 1 - Web Server 1	`general-website/`	**8090**	`senarath-sim-web`
SenNet online banking	DMZ 1 - Web Server 2	`online-banking/`	**8091**	`senarath-sim-bank`
MariaDB (two databases)	Database Servers tier	`db/`	**3307**	`senarath-sim-db`
SIEM log drop	NG SIEM Log Collector source	`siem-logs/`	-	(host bind mount)

Both applications are **fully functional** (real login sessions, real balance-moving transfers, persistent data) *and* intentionally vulnerable - the realism is what makes the generated telemetry useful for a SOC exercise. This is the same idea as DVWA / OWASP Juice Shop / IBM Altoro Mutual, wrapped in a realistic Sri Lankan bank.

---

2. Technology stack

Identical stack for both projects (kept simple, readable and easy to run on one Windows host).

Layer	Choice	Version	Notes
Language	Python	3.11 (`python:3.11-slim`)	
Web framework	Flask	3.0.3	Werkzeug **dev server with** `debug=True` (deliberate - exposes stack traces + interactive console)
Templating	Jinja2	(bundled with Flask)	Server-side rendered HTML
DB driver	PyMySQL	1.1.1	`DictCursor` , `autocommit=True`
Database	MariaDB	11.4	Two logical databases on one instance
Front-end	Hand-written HTML + CSS	-	No JS framework; vanilla CSS, navy + gold theme
Logging	Python `logging` + `RotatingFileHandler`	-	JSON-lines output (10 MB x 10 rotations per file)
Runtime	Docker + Docker Compose	-	3 services, 1 bridge network, 2 named volumes

Why Flask dev server and not gunicorn/nginx? On purpose. `debug=True` gives verbose error pages, the `/console` debugger, and no production hardening - all useful attack surface and telemetry for the lab. Do **not** reuse this configuration for anything real.

---

3. Repository structure

```
Public_facing_Webapp and website/
├── docker-compose.yml # db + both web apps (one host)
├── .env.example # WEB_URL / BANK_URL for your domain
├── README.md # quick-start + deploy + tunnel guide
├── PROJECT_DOCUMENTATION.md # (this file)
├── Logo/logo.png # Senarath Bank brand asset
│
├── db/init/ # auto-run by MariaDB on first boot (alphabetical)
│ ├── 00_user.sql # app DB user + grants (senarath_app / App@12345)
│ ├── 01_schema.sql # creates senarath_web + senarath_bank + tables
│ └── 02_seed.sql # fictional Sri Lankan data
│
├── general-website/ # PROJECT 1
│ ├── app.py # routes (vulns tagged "# [VULN: ...]")
│ ├── common.py # DB connection + JSON logging helpers
│ ├── requirements.txt Dockerfile
│ ├── templates/*.html # 21 Jinja templates (public site + CMS)
│ ├── static/css/style.css static/img/logo.png
│ └── logs/ # per-app log dir (also bind-mounted to siem-logs)
│
├── online-banking/ # PROJECT 2
│ ├── app.py common.py requirements.txt Dockerfile
│ ├── templates/*.html # 13 Jinja templates (SenNet app shell)
```

```
| ├── static/css/app.css static/img/logo.png
| └── logs/
|
└── siem-logs/ # <-- POINT THE NG SIEM COLLECTOR HERE
 ├── general-website/ # access.log, audit.log
 ├── online-banking/ # access.log, audit.log, auth.log
 └── db/ # general.log (MariaDB query log)
```

---

4. How it runs

```
cd "C:\Users\Laptop Outlet\Documents\Crypto_Projects\Crowdstike Event\Public_facing_Webapp and website"
docker compose up -d --build
```

- General website -> http://localhost:8090
- SenNet banking -> http://localhost:8091
- MariaDB -> localhost:3307

Service topology: all three containers share the `senarath` bridge network. The apps reach the database at `db:3306` (internal), while the host sees it on `3307`. Database data persists in the `dbdata` volume; uploaded CVs persist in `web_uploads`. All three services use `restart: unless-stopped`.

Reset to a clean, re-seeded state:

```
docker compose down -v # removes containers + the DB volume
docker compose up -d --build # re-runs the init scripts
```

---

5. Database

One MariaDB instance hosts two independent databases.

5.1 `senarath_web` (general website / CMS)

Table	Purpose
`cms_users`	Back-office logins (unsalted MD5)
`news_articles`	Editable news/announcements (body rendered unescaped)
`contact_messages`	Public contact-form submissions (shown unescaped in admin)
`newsletter_subscribers`	Email sign-ups
`branches`	8 branches island-wide
`careers` / `job_applications`	Job postings + applications (CV upload)
`audit_log`	In-DB mirror of security events

5.2 `senarath_bank` (online banking)

Table	Purpose
`customers`	12 fictional customers (name, NIC, email, phone, address)
`users`	Login accounts (customer/teller/admin; unsalted MD5; `failed_attempts` tracked but not enforced)
`accounts`	14 accounts (Savings/Current/FD, LKR balances)
`transactions`	History (debit/credit/transfer; `memo` rendered unescaped)
`beneficiaries`	Saved payees
`cards`	Masked card records
`support_tickets`	Customer tickets (rendered unescaped)
`login_log`	Every authentication attempt (brute-force telemetry)
`audit_log`	In-DB mirror of security events

5.3 Seeded data volume

12 customers · 14 accounts · 17 opening transactions · 4 beneficiaries · 3 cards · 2 tickets · 3 CMS users · 8 branches · 4 news articles · 4 job postings.

---

6. Credentials (lab use)

All passwords are intentionally weak and stored as **unsalted MD5**.

SenNet online banking (`:8091`)

Username	Password	Role
`kperera`	`password123`	customer (2 accounts)
`nsilva`	`123456`	customer
`sfernando`	`qwerty123`	customer (Rs. 2.5M FD)
`ajayawardena`	`senarath`	customer
`dbandara`	`dinesh123`	customer

`cwickrama`	`nimal2023`	customer
`irajapaksa`	`iloveyou`	customer
`tgunawardena`	`sunil@123`	customer
`msenanayake`	`abc12345`	customer (high balance)
`rdissanayake`	`letmein`	customer
`nherath`	`welcome1`	customer
`sweerasinghe`	`password`	customer
`teller1`	`teller123`	teller
`admin`	`admin123`	admin

General website CMS (`:8090/admin`)

Username	Password	Role
`admin`	`Admin@123`	superadmin
`nsenanayake`	`senarath2024`	admin
`editor1`	`password`	editor

Database (`:3307`)

`senarath_app` / `App@12345` (app user) · `root` / `Root@12345`

---

7. Endpoint inventory

7.1 General website (`general-website/app.py`)

Route	Method	Notes
`/`	GET	Home (latest news)
`/about` `/personal` `/business` `/loans` `/cards`	GET	Static marketing pages
`/branches`	GET	**SQLi** in `?city=`
`/news`	GET	News list
`/news/<slug>`	GET	**SQLi** in slug; **stored XSS** (body `|safe`)
`/search`	GET	**Reflected XSS** in `?q=`
`/careers`	GET	Job list
`/careers/apply`	POST	**Unrestricted file upload** (CV)
`/contact`	GET/POST	**Stored XSS** (message shown in admin)
`/subscribe`	POST	Newsletter
`/admin` `/admin/login`	GET/POST	**SQLi auth** + unsalted MD5 + no lockout + verbose error
`/admin/logout`	GET	
`/admin/dashboard`	GET	KPIs + recent audit events
`/admin/messages`	GET	**SQLi** in `?q=` ; **stored XSS** renders here
`/admin/messages.json`	GET	**Broken access control** (no auth)
`/admin/news/new` , `/admin/news/edit/<id>`	GET/POST	Create/edit article (stores raw HTML)
`/admin/users`	GET	**Sensitive data exposure** (shows hashes)
`/robots.txt`	GET	**Info disclosure** (advertises `/admin` , `/backup.sql` , `/debug`)
`/backup.sql`	GET	**Exposed DB backup**
`/debug`	GET	**Config/secret disclosure**
`/uploads/` , `/uploads/<file>`	GET	**Directory listing** + serves uploaded files (no auth)

7.2 SenNet online banking (`online-banking/app.py`)

Route	Method	Notes
`/`	GET	Redirects to dashboard/login
`/login`	GET/POST	**SQLi auth** + MD5 + no lockout + **user enumeration** + verbose error
`/logout`	GET	
`/dashboard`	GET	Account summary (memo rendered unescaped)
`/accounts`	GET	Own accounts
`/account/<account_no>`	GET	**IDOR** - any account by number, no ownership check
`/transfer`	GET/POST	**IDOR source**, **no balance/positive-amount check**, **stored XSS** memo, **no CSRF**
`/payments`	GET/POST	Bill payment (debits account)
`/beneficiaries`	GET/POST	**Stored XSS** (nickname/payee)

`/statements`	GET	**IDOR** - statement for any account in `?account=`
`/profile`	GET/POST	**Stored XSS** (address); no re-auth
`/support`	GET/POST	**Stored XSS** (subject/body)
`/admin`	GET	**Broken access control** - any logged-in user; shows all users/hashes/accounts
`/api/customer/<id>`	GET	**Unauthenticated PII** (IDOR)
`/api/accounts`	GET	**Unauthenticated full account dump**
`/debug`	GET	**Config/secret disclosure**
`/robots.txt`	GET	Info disclosure

8. Vulnerability catalogue (attack surface -> OWASP -> telemetry)

Every item is deliberate and tagged `# [VULN: ...]` in the source. The telemetry column is what the SIEM should be able to see when the weakness is exercised - that is the detection-engineering value.

8.1 SenNet online banking

#	Weakness	Location	OWASP 2021	Telemetry produced
1	SQL injection in login	`/login`	A03	`auth.log` failures; `audit.log` `attack_suspected` (`sqli`); SQL errors
2	Unsalted MD5, no lockout	login / DB	A07	repeated `authentication/failure` from one IP
3	Username enumeration	`/login` error text	A07	distinguishable "User not found" vs "Incorrect password"
4	IDOR - view any account	`/account/<no>`	A01	`access_control / idor_account_view` (high) with attacker+victim IDs
5	IDOR - any statement	`/statements?account=`	A01	`access_control / idor_statement` (high)
6	Transfer from non-owned account	`/transfer`	A01	`access_control / idor_transfer_source` (critical) + `transaction/funds_transfer`
7	No balance / no positive-amount check	`/transfer`	A04	overdrafts & negative amounts; anomalous `funds_transfer`
8	Broken access control to admin	`/admin`	A01	`access_control / admin_privilege_escalation` (critical)
9	Unauthenticated PII API	`/api/customer/<id>`	A01/A02	`info_disclosure / api_customer` (high)
10	Unauthenticated account dump	`/api/accounts`	A01	`info_disclosure / api_accounts_dump` (critical)
11	Stored XSS	transfer memo, beneficiary, ticket	A03	`attack_suspected` (`xss`) on submit
12	No CSRF protection	transfers, profile	A01	state-changing requests with foreign `Referer`
13	Config / secret disclosure	`/debug`	A05	`info_disclosure / debug_endpoint`
14	Weak session key, insecure cookies	app config	A05/A07	forgeable sessions (no HttpOnly/Secure/SameSite)
15	Verbose errors / debug console	all	A05	stack traces, Werkzeug `/console`
16	Missing security headers	all responses	A05	no CSP/HSTS/X-Frame-Options/nosniff

8.2 General website / CMS

#	Weakness	Location	OWASP 2021	Telemetry produced
1	SQL injection	`/branches?city=`, `/news/<slug>`, CMS login, `/admin/messages?q=`	A03	`sqli` tags, `sql_error` events, verbose errors
2	Stored XSS	contact message (admin inbox), news body	A03	`xss` tag on submit; fires in `/admin/messages`
3	Reflected XSS	`/search?q=`	A03	`xss` tag in query string
4	Broken access control	`/admin/messages.json` (no auth)	A01	`access_control / messages_export` (high)
5	Sensitive data exposure	`/admin/users` (hashes), `/backup.sql`	A02	`info_disclosure / backup_sql_accessed` (critical)
6	Unrestricted file upload	`/careers` -> served from `/uploads/`	A05/A08	`file_upload` (medium); uploads dir listing
7	Info disclosure	`/robots.txt`, `/debug`	A05	`info_disclosure` events
8	Missing security headers	all responses	A05	no CSP/HSTS/X-Frame-Options/nosniff
9	Weak session key, insecure cookies	app config	A05/A07	forgeable sessions
10	Verbose errors / debug console	all	A05	stack traces, Werkzeug `/console`

9. Logging & SIEM integration

9.1 Where logs are written

Each app logs to `/app/logs` inside its container, which is **bind-mounted to the host** so the collector can tail it. Audit events are additionally **mirrored into DB tables** and **echoed to stdout** (so `docker logs` and the host Falcon sensor also see them).

Source	Container path	Host path	Contents
Website access	`/app/logs/access.log`	`siem-logs/general-website/access.log`	one JSON line per HTTP request
Website audit	`/app/logs/audit.log`	`siem-logs/general-website/audit.log`	one JSON line per security event
Banking access	`/app/logs/access.log`	`siem-logs/online-banking/access.log`	per-request
Banking audit	`/app/logs/audit.log`	`siem-logs/online-banking/audit.log`	security events
Banking auth	`/app/logs/auth.log`	`siem-logs/online-banking/auth.log`	every login attempt

MariaDB query log	`/var/log/mysql/general.log`	`siem-logs/db/general.log`	raw SQL that reached the DB
DB audit mirror	-	table `senarath_web.audit_log` / `senarath_bank.audit_log`	queryable copy of audit events
DB auth mirror	-	table `senarath_bank.login_log`	queryable copy of auth attempts
stdout	-	`docker logs senarath-sim-web` / `-bank`	audit (+ auth) events

Rotation: each file rotates at **10 MB**, keeping **10** backups.

9.2 Audit event schema (one flat JSON object per line)

```
{
"@timestamp": "2026-10-28T16:40:12.123456+00:00",
"app": "senarath-bank",
"log_type": "audit",
"event_type": "access_control",
"severity": "critical",
"action": "idor_transfer_source",
"outcome": "success",
"username": "kperera",
"src_ip": "203.0.113.45",
"http_method": "POST",
"path": "/transfer",
"detail": "transfer initiated from non-owned account 1001000011",
"amount": 500000.0,
"from_account": "1001000011",
"to_account": "1001000099",
"reference": "TRF963632002"
}
```

Access log fields: `@timestamp, app, log_type=access, src_ip, method, path, query, status, bytes, referer, user_agent, username, duration_ms`. **Auth log fields:** `@timestamp, app, log_type=auth, event_type=authentication, username, outcome, src_ip, user_agent, detail, severity`.

9.3 Event taxonomy (values to build detections on)

Field	Values
`event_type`	`authentication`, `attack_suspected`, `access_control`, `transaction`, `info_disclosure`, `sql_error`, `file_upload`, `form_submit`, `content_change`, `account_change`, `support`, `admin_access`, `system`
`severity`	`info`, `medium`, `high`, `critical`, `warn`
`outcome`	`success`, `failure`, `error`, `flagged`
`attack_tags`	`sqli`, `xss`, `path_traversal`

> **Source IP note:** `src_ip` honours the `X-Forwarded-For` header. When you front the lab with a proxy/tunnel (see README §7), set that header at the edge so the SIEM records the real attacker IP. It is also trusted blindly - which is realistic and itself spoofable.

9.4 Collector configuration (example)

Point the **NG SIEM Log Collector (on-prem)** at the host folder and parse as JSON. For the CrowdStrike LogScale / NG SIEM path, a Falcon Log Collector file source:

```
sources:
senarath-weblogs:
type: file
include: ["/data/senarath/siem-logs/**/*.log"]
parser: json
```

9.5 Starter detections

Scenario	Logic
Brute force / password spray	`event_type=authentication outcome=failure` grouped by `src_ip` over a time window
Credential stuffing success after failures	failures then `outcome=success` from same `src_ip`
IDOR / privilege escalation	`event_type=access_control severity in (high, critical)`
Injection attempts	`event_type=attack_suspected` or `attack_tags` contains `sqli` / `xss`
Sensitive data exposure	`event_type=info_disclosure` (watch `backup_sql_accessed`, `api_accounts_dump`)
Anomalous transfer	`action=funds_transfer` with large or negative `amount`, or `idor_transfer_source`
Web shell upload	`action=cv_uploaded` followed by a GET to `/uploads/<same file>`

---

10. Deployment & public exposure (summary)

Full steps, nginx config and comparison table are in **README.md §6-7**. In short:

1. Run `docker compose up -d --build` on the Windows host (Docker Desktop, WSL2).
2. Set `WEB_URL` / `BANK_URL` in `.env` to your public domain so cross-links resolve.
3. Expose it **without Cloudflare** (its WAF/encrypted tunnel would filter the very attacks you want): * **A - VPS + `frp` reverse tunnel + your domain** *(recommended)* - stable domain, raw attacks reach the app, real client IP preserved via nginx `X-Forwarded-For`, host stays off the open net. * **B - ngrok** - quickest for a short demo. * **C - router port-forward + DuckDNS/No-IP** - most "raw"/realistic edge, cheapest, but exposes your own network (check you are not behind CGNAT).

---

11. Safe operation

- **Isolate it.** Dedicated VLAN/host, no route to any real asset. An exposed vulnerable box *will* be compromised and can be used to pivot - containment is the exercise.
- **No real data, ever.** Fake customers, fake money, lab-only passwords; don't reuse passwords.
- **Watch egress.** Your Falcon sensor + SIEM should alert on outbound from these containers - a good detection and your early warning the box was taken over.
- **Rebuild often.** `docker compose down -v && docker compose up -d --build` between exercises.
- **Keep the "SIMULATION - NOT A REAL BANK" banner** on every page so no human mistakes it for a real bank or enters real details.
- **Take it down** when the exercise is over.

Senarath Bank is a fictional institution created solely for cybersecurity simulation and SIEM detection training. Any resemblance to a real bank is coincidental.